

Функция - отдельно оформленный блок кода, который:

1. выполняет некоторое **законченное** по смыслу **действие**
2. имеет **уникальное имя** или его аналог
3. состоит из **набора инструкций**
4. может быть **многократно выполнен** (вызов функции)
5. имеет свойства для **обмена данными** с остальным кодом (параметры, результат)

Тоже функции: +, -, >>

Объявление

1. Тип результата
2. Имя функции
3. Список формальных параметров

Определение

Определение происходит после объявления, в фигурных скобках размещается набор инструкций

- Функции должны быть объявлены до их вызова
- Функции могут быть определены в коде ниже, чем их вызов

Часто сперва объявляют функцию, а определяют ее внизу, после main'a

Вызов

1. Имя функции

2. Список фактических параметров

3. Точка с запятой

Параметры

1. Формальные (при объявлении)

2. Фактические (при вызове)

Формальные

1. Указывают только тип и имя

2. Фактическим значением может быть любое выражение подходящего типа

3. При выполнении создается локальная переменная (копия) с переданным значением

Для того чтобы изменить значение переданного параметра, его можно передать по ссылке

Для параметра можно задать значение по умолчанию. Такие параметры должны находиться в конце списка параметров

\

Константные параметры

```
// При модификаторе const нельзя менять значения
// параметров, переданных в функцию
int plus(const int a, int b) {
    a = a * 10; // Error
    return a + b;
}
```

Указатель

- Для параметров-указателей адреса передаются как значения, то есть создается локальный указатель на адрес
- По этому адресу можно менять данные

Одномерный массив

Параметр-массив может быть использован только для встроеного массива

При таком подходе в функцию передается не массив, а указатель на массив (!)

```
void func(int a[], int n) {  
    size_t length = sizeof(a) / sizeof(int); //  
    Error, поэтому передаем длину массива (n)  
    ...  
}
```

Параметры-указатели могут быть использованы как для встроённых, так и для динамических массивов

```
void func(int* a, int n) {  
    ...  
}
```

Многомерный массив

Для параметра в виде многомерного массива все размерности кроме первой должны быть указаны явно, в форме константного выражения

```
void func(int a[][3], int n) {  
    ...  
}
```

Указатель на многомерный массив

```
void func(int** array, int n, int m) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            ...  
        }  
    }  
}
```

Переменное количество параметров

```
void func (string str, int n, ...) {  
    va_list args; // Создаем указатель на список  
аргументов  
    va_start(args, n); // Устанавливаем указатель на  
аргумент n  
    va_arg(args, int); // Указываем аргумент и его  
тип  
    va_end(args); // Завершаем обработку аргументов  
    ...  
}
```

Перегрузка функций

Перегрузка функций - создание функций с одним именем, но с разными параметрами (сигнатурами)

Особенности

1. Перегрузка является одной из реализаций полиморфизма
2. Если при перегрузке используются параметры с приводимыми типами (`double` и `int`) есть возможность получить ошибку: компилятор не поймет какую функцию вызвать